

Label noise robustness of a small CNN on 8x8 digits: cross-entropy, label smoothing, mixup and small-loss selection

Pollux-19N
STEM seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/label-noise-augmentation-small-cnn>

Abstract—We compare four training objectives for a small CNN under symmetric label noise on the scikit-learn 8×8 digits: cross-entropy (CE), label smoothing (LS), mixup, and a one-network small-loss selection rule in the spirit of co-teaching, at 0, 20, 40 and 60% noise with 5 seeds each, all reimplemented with identical architecture, schedule and data. Besides clean test accuracy, we measure memorisation on the corrupted training examples (the fraction fitted to the wrong label minus the fraction predicted as the true label). CE memorises nearly all corrupted labels and drops from 0.991 to 0.709 at 40% noise. LS does not help (0.668 at 40%). Mixup with $\alpha=1$ helps modestly (0.762) yet still memorises 89% of corrupted labels. Small-loss selection, given the true noise rate, is best at 20–60% noise (0.896 at 40%, 0.771 at 60%). Sensitivity sweeps show that under-estimating the noise rate erodes its gain, that over-estimating it does not, and that larger mixup α keeps improving. Test accuracy logged at earlier epochs shows LS above CE and mixup close to small-loss before memorisation sets in, so these rankings are specific to final-epoch evaluation. The study is small (one tiny dataset, untuned baselines, oracle forget rate, final-epoch evaluation), so the ranking should be read as a controlled small-scale observation, not as a general result.

Index Terms—label noise, memorisation, mixup, label smoothing, sample selection, small CNN, digits

I. Introduction

Deep networks can fit arbitrary labels, including random ones, which makes memorisation of mislabelled training examples a central worry when labels are noisy [1]. At the same time, networks tend to learn examples with consistent patterns before they memorise noisy ones [2], and this ordering is the basis of many remedies: curricula and sample selection by small loss [3]–[5], regularisers that exploit early learning [6], and data-independent regularisers such as mixup [7] and label smoothing [8]. Comparisons run at larger scale and with per-method tuning make it hard to tell how much of a method’s benefit comes from the method and how much from the recipe.

This paper is a deliberately small, controlled comparison of four training objectives on one tiny dataset: the scikit-

learn 8×8 digits. Using one fixed CNN, optimiser, schedule and data split, we compare plain cross-entropy (CE), label smoothing (LS), mixup, and a one-network small-loss selection rule in the spirit of co-teaching [4], at symmetric noise rates of 0, 20, 40 and 60%. Besides clean test accuracy we measure memorisation directly on the corrupted training examples: the fraction predicted as their wrong label (`mem_rate`), the fraction predicted as their true label (`recover_rate`), and their difference (`mem_gap`).

We pre-registered five hypotheses (H1–H5, Sec. IV) and report each as supported, refuted or inconclusive. We claim no new algorithm. The contributions are (i) an equal-budget, 5-seed comparison with paired tests, (ii) a direct memorised-versus-clean-label measure, and (iii) sensitivity sweeps for the hyperparameter of each method. All numbers come from CPU runs in the accompanying repository; the study is small and its limits are listed in Sec. VII.

II. Related Work

Memorisation. Zhang et al. [1] showed that standard networks fit random labels, so capacity alone does not prevent memorisation. Arpit et al. [2] found that networks fit simple, shared patterns first and noisy examples later, and that regularisation and data affect this differently. Patel and Sastry [9] study how the choice of loss function changes memorisation of noisy labels, which is close to our question but with different losses.

Sample selection. MentorNet [3] learns a curriculum that down-weights probable outliers; Co-teaching [4] trains two networks that each pass their small-loss examples to the other, with a forget rate that follows the noise rate and increases over training; DivideMix [5] splits data into clean and noisy parts with a mixture model and continues semi-supervised. Our selection baseline is the one-network, per-batch variant of the small-loss rule; it is a reimplement and omits the cross-update that gives Co-teaching its name, so it should not be read as a reproduction of that method.

Regularisation. Early-learning regularisation (ELR) [6] prevents memorisation by steering towards early-training

Pollux-19N is the codename of an autonomous study, not a person. Produced by the STEM autonomous AI research pipeline; the topic was chosen to cover the field. Every reported number comes from logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

predictions. Mixup [7] trains on convex combinations of examples and labels and was reported to reduce memorisation of corrupted labels. Label smoothing [8] replaces hard targets by a mixture with the uniform distribution. Lukasik et al. [10] analyse whether it mitigates label noise, relating it to a form of loss correction, while Wei et al. [11] report that its advantage vanishes in a high-noise regime. Our experiments test these two statements on a tiny dataset.

III. Method

Let $\{(x_i, \tilde{y}_i)\}_{i=1}^n$ be the training set with observed labels $\tilde{y}_i \in \{1..K\}$, $K=10$, and let $f_\theta(x)$ be the logits of the network with softmax p_θ . All systems use the same network and optimiser and differ only in the per-batch loss $\mathcal{L}$ on a mini-batch B .

Symmetric noise. With rate r , each training label is kept with probability $1 - r$ and otherwise replaced by a uniformly chosen different class, $\tilde{y}_i = y_i$ w.p. $1 - r$ and $\tilde{y}_i \sim U(\{1..K\} \setminus y_i)$ w.p. r . Test labels are clean.

CE. $\mathcal{L}_{\text{CE}} = \frac{1}{|B|} \sum_{i \in B} -\log p_\theta(\tilde{y}_i | x_i)$.

Label smoothing (reimplemented) [8]. With target $q_\epsilon = (1 - \epsilon)e_{\tilde{y}} + \epsilon/K$ (implemented by the framework option, which uses ϵ/K on all classes): $\mathcal{L}_{\text{LS}} = \frac{1}{|B|} \sum_{i \in B} -\sum_k q_\epsilon(k) \log p_\theta(k | x_i)$, $\epsilon = 0.1$ in the main grid.

Mixup (reimplemented) [7]. Draw $\lambda \sim \text{Beta}(\alpha, \alpha)$ per batch, a permutation π of the batch, $\bar{x}_i = \lambda x_i + (1 - \lambda)x_{\pi(i)}$ and $\mathcal{L}_{\text{mix}} = \frac{1}{|B|} \sum_i [\lambda \ell(f_\theta(\bar{x}_i), \tilde{y}_i) + (1 - \lambda) \ell(f_\theta(\bar{x}_i), \tilde{y}_{\pi(i)})]$, with ℓ the CE; $\alpha = 1$ in the main grid.

Small-loss selection, one network (reimplemented, co-teaching style [4]). With per-example losses ℓ_i and forget rate r_t at epoch t , keep the $\max\{1, \text{round}((1 - r_t)|B|)\}$ examples with the smallest ℓ_i in the batch, $S_t(B)$, and minimise $\mathcal{L}_{\text{SL}} = \frac{1}{|S_t(B)|} \sum_{i \in S_t(B)} \ell_i$ (selection is not differentiated). The forget rate is $r_t = 0$ for the first $T_w=10$ epochs, then $r_t = r \cdot \min\{1, (t - T_w + 1)/T_r\}$ with $T_r=10$. In the main grid r is the true noise rate; this is an oracle assumption that a practitioner does not have, and Sec. VI varies it. Because $r=0$ at 0% noise, this system then reduces exactly to CE.

Memorisation metrics. Let $C = \{i : \tilde{y}_i \neq y_i\}$ be the corrupted training examples and $\hat{y}_i$ the final prediction (eval mode, no augmentation). $\text{mem_rate} = \frac{1}{|C|} \sum_{i \in C} \mathbb{I}[\hat{y}_i = \tilde{y}_i]$, $\text{recover_rate} = \frac{1}{|C|} \sum_{i \in C} \mathbb{I}[\hat{y}_i = y_i]$, $\text{mem_gap} = \text{mem_rate} - \text{recover_rate} \in [-1, 1]$: +1 means every corrupted label is memorised, -1 means every corrupted example is classified by its true class. At 0% noise $C = \emptyset$ and the three are defined as 0.

IV. Experimental Setup

Hypotheses (fixed before the runs). H1: CE memorises ($\text{mem_rate} > 0.9$) and loses more than 10 points of test accuracy at 40% and 60% noise relative to 0% noise. H2: label smoothing ($\epsilon=0.1$) gains less than 3 points over CE at 40% noise. H3: mixup ($\alpha=1$) lowers mem_rate and

raises test accuracy over CE at 20–60% noise. H4: small-loss selection with the true noise rate is the most accurate system at 40% and 60% noise, with the most negative mem_gap , but is worse than CE at 0% noise. H5: small-loss selection depends on the assumed forget rate: underestimating the noise rate loses most of the gain.

Data. scikit-learn digits (1,797 images, 8×8 , $K=10$), pixels divided by 16. One fixed stratified split (`random_state=1234`): 1,078 training and 719 test images; the split does not change with the seed. Symmetric noise is drawn on the training labels only, with a seed-dependent generator. Each noise rate is one task (`digits_noise0/20/40/60`). There is no validation set and no early stopping; all hyperparameters were fixed a priori and the final-epoch model is evaluated, so memorisation after long training is what is measured. The sweeps of Sec. VI are evaluated on the test split and are reported as sensitivity analyses; they were not used to choose the main settings.

Model and training. Conv(1→16)–BN–ReLU, Conv(16→32)–BN–ReLU–MaxPool, Conv(32→64)–BN–ReLU–MaxPool, FC(256→64)–ReLU–FC(64→10), all convolutions 3×3 (about 41k parameters). SGD, momentum 0.9, learning rate 0.05 with cosine decay, weight decay $5 \cdot 10^{-4}$, batch size 64, 60 epochs, no data augmentation other than mixup where used. These settings are identical for all systems; the only per-system hyperparameters are $\epsilon=0.1$ (LS), $\alpha=1$ (mixup) and the schedule of Sec. III (small-loss). No hyperparameter was tuned.

Protocol. 4 systems $\times$ 4 noise rates $\times$ 5 seeds (0–4) = 80 main runs; the seed sets the noise draw, initialisation and batch order. The sensitivity runs use 40% noise and 5 seeds each: 4 mixup α values, 4 LS ϵ values, 5 assumed forget rates and one no-warm-up variant (70 runs). Tables report mean $\pm$ standard deviation over seeds; p -values are paired two-sided t -tests over the 5 seeds (the seed fixes the noise draw, so the systems see the same corrupted labels). With $n=5$ these tests have little power and are indicative only; no multiple-comparison correction is applied.

Compute. CPU only, 2 threads, a shared machine; a run takes about 12s on an idle core and a median of 23s on the loaded machine used (the two slowest runs took 334s and 367s from contention); total about 80 CPU-minutes for 150 runs. All baselines are reimplemented here in one script with identical code paths for data, model and evaluation.

V. Results

Table I gives the main grid (5 seeds, mean $\pm$ std), Fig. 1 the same trends, and Table II the paired differences to CE, written as CE minus system (a negative test-accuracy entry means the system is above CE; a positive mem_rate entry means it memorises less). At 0% noise there are no corrupted examples, so the memorisation columns are 0 by definition.

TABLE I

Main grid: final-epoch clean test accuracy and memorisation of the corrupted training labels (mean $\pm$ std over 5 seeds; bold = best, underline = second best within a noise rate). Small-loss uses the true noise rate.

Method	Task	test_acc $\uparrow$	mem_rate $\downarrow$	recover_rate $\uparrow$	mem_gap $\downarrow$	n
CE	digits_noise0	0.991 $\pm$ 0.003	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
Mixup	digits_noise0	0.993 $\pm$ 0.002	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
Label smoothing	digits_noise0	0.995 $\pm$ 0.001	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
Small-loss (1 net)	digits_noise0	0.991 $\pm$ 0.003	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
CE	digits_noise20	0.897 $\pm$ 0.017	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5
Mixup	digits_noise20	0.921 $\pm$ 0.013	0.919 $\pm$ 0.019	0.079 $\pm$ 0.019	0.839 $\pm$ 0.038	5
Label smoothing	digits_noise20	0.894 $\pm$ 0.012	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5
Small-loss (1 net)	digits_noise20	0.943 $\pm$ 0.017	0.419 $\pm$ 0.056	0.550 $\pm$ 0.063	-0.132 $\pm$ 0.119	5
CE	digits_noise40	0.709 $\pm$ 0.035	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5
Mixup	digits_noise40	0.762 $\pm$ 0.030	0.892 $\pm$ 0.006	0.073 $\pm$ 0.016	0.819 $\pm$ 0.020	5
Label smoothing	digits_noise40	0.668 $\pm$ 0.028	0.999 $\pm$ 0.002	0.000 $\pm$ 0.000	0.999 $\pm$ 0.002	5
Small-loss (1 net)	digits_noise40	0.896 $\pm$ 0.018	0.280 $\pm$ 0.037	0.649 $\pm$ 0.048	-0.370 $\pm$ 0.084	5
CE	digits_noise60	0.436 $\pm$ 0.039	0.995 $\pm$ 0.008	0.001 $\pm$ 0.002	0.994 $\pm$ 0.010	5
Mixup	digits_noise60	0.523 $\pm$ 0.032	0.853 $\pm$ 0.049	0.078 $\pm$ 0.020	0.775 $\pm$ 0.066	5
Label smoothing	digits_noise60	0.410 $\pm$ 0.041	0.994 $\pm$ 0.007	0.001 $\pm$ 0.002	0.993 $\pm$ 0.009	5
Small-loss (1 net)	digits_noise60	0.771 $\pm$ 0.035	0.230 $\pm$ 0.025	0.625 $\pm$ 0.035	-0.395 $\pm$ 0.059	5

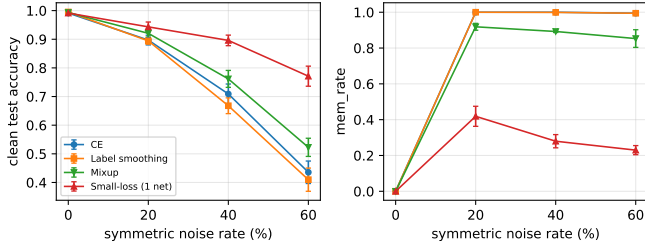


Fig. 1. Test accuracy (left) and mem_rate (right) against the symmetric noise rate; mean $\pm$ std over 5 seeds.

TABLE II

Differences to CE over the same 5 seeds (CE minus system, from the registry comparison `rh compare -ref CE`) and paired t -test p -value for test accuracy; $n=5$, no multiple-comparison correction.

Noise	System	acc: CE-sys.	mem_rate: CE-sys.	p_{acc}
20%	LS	0.003	0.000	0.5740
20%	Mixup	-0.024	0.081	0.04288
20%	Small-loss	-0.046	0.581	0.0007546
40%	LS	0.041	0.001	0.01411
40%	Mixup	-0.053	0.108	0.0002774
40%	Small-loss	-0.187	0.720	6.857×10^{-5}
60%	LS	0.026	0.000	0.09484
60%	Mixup	-0.087	0.142	0.001751
60%	Small-loss	-0.335	0.765	4.251×10^{-5}

Evaluation epoch. The run logs record test accuracy every 5 epochs. These logged curves (summarised over the 5 seeds by `analysis/make_curves.py`) show that the final-epoch ranking is specific to end-of-training evaluation: at 40% noise the mean best logged checkpoint of LS exceeds that of CE and that of mixup nearly matches that of small-loss before memorisation sets in, while small-loss is the only system whose final accuracy stays close to its best checkpoint. The checkpoint accuracies live in the run logs and are not metrics of the run registry, so we state only

their ordering and quote no values. The best checkpoint is the maximum on the test split and is an upper bound, not a protocol a practitioner could follow.

H1 (CE memorises): supported. CE fits essentially all corrupted labels (mem_rate 1.000 at 20% and 40% noise, 0.995 at 60%) even though the network has only about 41k parameters and weight decay. Its test accuracy falls from 0.991 at 0% noise to 0.709 at 40% and 0.436 at 60%, i.e. by far more than the 0.10 of H1 (at 20% noise it already falls to 0.897). Final-epoch accuracy is measured with no early stopping, so these numbers reflect the end of training, not the best epoch.

H2 (label smoothing gives no material gain): supported, and it is worse than that. With $\epsilon=0.1$, LS is indistinguishable from CE at 20% (paired $p=0.5740$), and is below CE at 40% (CE minus LS 0.041, $p=0.01411$, uncorrected) and at 60% (CE minus LS 0.026, $p=0.09484$); mem_rate is unchanged (about 1.0). LS is the best system only at 0% noise (0.995 vs 0.991 for CE; CE minus LS is -0.4 points over 5 seeds, which we do not interpret). The ϵ sweep (Table III) gives test accuracy from 0.662 to 0.693 over $\epsilon \in [0.05, 0.6]$ against CE's 0.709, with no value reducing mem_rate, so within this range the conclusion does not depend on ϵ . This contrasts with Lukasik et al. [10], who find label smoothing competitive under label noise; our setting (tiny data, a network that memorises completely, final-epoch evaluation) differs from theirs, and the logged curves show that LS does exceed CE before memorisation sets in (mean best logged checkpoint at 40%), so the disagreement is about end-of-training behaviour.

H3 (mixup helps): supported at $\alpha=1$, with a caveat on mechanism. Mixup beats CE at 40% and 60% noise clearly and at 20% only suggestively (the 20% result rests on an uncorrected p of 0.04288 and one seed goes the other way) (CE minus mixup: -0.024, -0.053, -0.087; paired p 0.04288, 0.0002774, 0.001751) and lowers mem_rate

TABLE III

Sensitivity at 40% noise (mean $\pm$ std over 5 seeds). CE reference: test_acc 0.709, mem_rate 1.000.

System	Setting	test_acc	mem_rate	mem_gap
Mixup	$\alpha=0.2$	0.709 $\pm$ 0.037	0.984 $\pm$ 0.010	0.976 $\pm$ 0.015
Mixup	$\alpha=0.5$	0.735 $\pm$ 0.022	0.958 $\pm$ 0.020	0.928 $\pm$ 0.033
Mixup	$\alpha=1$ (main)	0.762 $\pm$ 0.030	0.892 $\pm$ 0.006	0.819 $\pm$ 0.020
Mixup	$\alpha=2$	0.794 $\pm$ 0.026	0.718 $\pm$ 0.013	0.486 $\pm$ 0.029
Mixup	$\alpha=4$	0.849 $\pm$ 0.038	0.426 $\pm$ 0.071	-0.085 $\pm$ 0.156
LS	$\epsilon=0.05$	0.662 $\pm$ 0.042	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
LS	$\epsilon=0.1$ (main)	0.668 $\pm$ 0.028	0.999 $\pm$ 0.002	0.999 $\pm$ 0.002
LS	$\epsilon=0.2$	0.676 $\pm$ 0.034	0.999 $\pm$ 0.001	0.999 $\pm$ 0.002
LS	$\epsilon=0.4$	0.682 $\pm$ 0.036	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
LS	$\epsilon=0.6$	0.693 $\pm$ 0.030	1.000 $\pm$ 0.001	0.999 $\pm$ 0.002
Small-loss	rate=0.1	0.733 $\pm$ 0.029	0.875 $\pm$ 0.005	0.778 $\pm$ 0.009
Small-loss	rate=0.2	0.774 $\pm$ 0.023	0.685 $\pm$ 0.015	0.418 $\pm$ 0.027
Small-loss	rate=0.3	0.828 $\pm$ 0.016	0.509 $\pm$ 0.015	0.082 $\pm$ 0.034
Small-loss	rate=0.4 (main)	0.896 $\pm$ 0.018	0.280 $\pm$ 0.037	-0.370 $\pm$ 0.084
Small-loss	rate=0.5	0.918 $\pm$ 0.016	0.162 $\pm$ 0.010	-0.612 $\pm$ 0.026
Small-loss	rate=0.6	0.921 $\pm$ 0.019	0.116 $\pm$ 0.021	-0.702 $\pm$ 0.056
Small-loss	no warm-up/ramp, rate=0.4	0.924 $\pm$ 0.019	0.146 $\pm$ 0.040	-0.649 $\pm$ 0.091

(0.919, 0.892, 0.853 against about 1.0). It is nevertheless far from solving the problem: it still memorises 85–92% of the corrupted labels and recovers the true class for only 7–8%, so its accuracy gain does not come from avoiding memorisation; we conjecture it comes from the smoothness of the memorised solution, but no experiment here tests that mechanism. Its gain grows with the noise rate, as with the sample-selection rule.

H4 (small-loss is best at 40/60% but loses at 0%): half supported. With the true noise rate supplied, small-loss selection is the most accurate system at 20, 40 and 60% (0.943, 0.896, 0.771; CE minus small-loss is -0.187 and -0.335 at 40 and 60%, paired p 6.857×10^{-5} and 4.251×10^{-5}) and the only one with a negative mem_gap (it classifies more corrupted training images by their true class than by the wrong one: -0.370 at 40% and -0.395 at 60%). The second clause is not testable as stated: at 0% noise the forget rate is 0, so the system is numerically identical to CE (same seeds, identical accuracy 0.991), and it does not lose to CE. The cost of selection should instead be read from where it fails: at 60% noise it still ends at 0.771, well below its own 0% accuracy of 0.991, and at 20% noise it fits 42% of the corrupted labels (mem_rate 0.419) because the forget rate ramps only after a warm-up in which the network is trained on all examples.

Failure cases. (i) At 60% noise every system is far from the clean-label accuracy: CE 0.436, LS 0.410, mixup 0.523, small-loss 0.771. (ii) Small-loss selection relies on knowing the noise rate; this is not available in practice and Sec. VI shows what happens when it is wrong. (iii) The per-seed spread of mem_gap for small-loss at 20% noise is large (-0.132 ± 0.119): the same recipe sometimes ends with the memorised and sometimes with the clean label dominant.

VI. Ablations and Analysis

All sensitivity runs use 40% noise and the same 5 seeds; the main-setting rows are the main-grid runs (not re-run). The sweeps are evaluated on the test split and are not used to revise the main results.

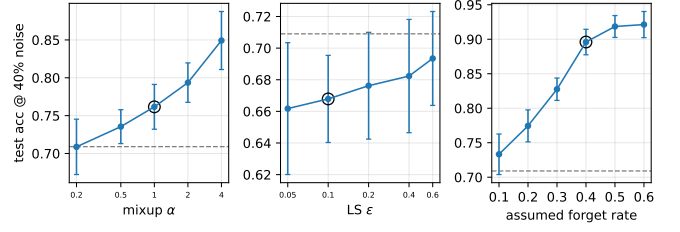


Fig. 2. Test accuracy at 40% noise against the method hyperparameter (mean $\pm$ std, 5 seeds). Ring: main setting; dashed line: CE.

H5 (“under-estimating the noise rate loses most of the gain”): partly supported, and the sign of the error matters. Under-estimating the noise rate costs accuracy monotonically; most of the gain over CE is lost at assumed rates 0.1 and 0.2, but not at 0.3: assumed rates 0.1, 0.2, 0.3 give 0.733, 0.774, 0.828 against 0.896 at the true rate 0.4, and mem_rate rises from 0.280 to 0.875 at 0.1, close to CE (0.709/1.000). Over-estimating does not hurt here: rates 0.5 and 0.6 give 0.918 and 0.921, higher than the true rate, within roughly one standard deviation of each other. Discarding more than the true fraction removes some clean examples but, on a dataset this redundant (1,078 training examples of 10 well-separated classes), still leaves enough clean ones. This is an observation for this dataset only; it would not be expected to hold when classes are rare or hard. We did not sweep rates above 0.6.

Warm-up and ramp. Removing the 10-epoch warm-up and the 10-epoch ramp (selection at the full rate from epoch 0) gives 0.924 against 0.896 for the main setting (Table III) and mem_rate 0.146 against 0.280. The main schedule was fixed before the runs. Both variants use the same five seeds and noise draws and are evaluated on the test split; we report the two means only and no test, because the registry comparison works within one run group and these rows are in two. The direction is consistent with memorisation having started during the warm-up. This also means the main-grid small-loss numbers are conservative with respect to this component, not tuned.

Mixup strength. Test accuracy rises monotonically with α (0.709, 0.735, 0.762, 0.794, 0.849 for $\alpha = 0.2, 0.5, 1, 2, 4$) while mem_rate falls from 0.984 to 0.426; at $\alpha = 4$ the mem_gap is -0.085 ± 0.156 , i.e. roughly balanced. The mixup number in the main grid ($\alpha=1$) is therefore a weak setting for mixup on this task; the main comparison was not re-done with $\alpha=4$, since choosing it after seeing test results would be tuning on the test split. At $\alpha=4$ mixup (0.849) is still below small-loss at the true rate (0.896) but this is a different comparison from the pre-registered one, and the sweep did not include the matched sweep over the small-loss schedule (T_w , T_r) that would be needed for a fair tuned comparison.

Label smoothing strength. Accuracy changes only from 0.662 to 0.693 over a 12-fold range of ϵ with mem_rate

pinned at 1.0, so the smoothing target does not prevent memorisation in this regime; this is in line with the report that its benefit vanishes at high noise [11], though our setting differs.

VII. Limitations

- Scale. One tiny dataset (1,078 training images of 8×8 digits), one 41k-parameter CNN, 60 epochs, 5 seeds, one train/test split. Digits are easy and redundant; with 5 seeds, p -values are indicative and uncorrected. Conclusions need not carry over to CIFAR-scale data or deeper networks.
- Noise model. Only symmetric class-conditional noise, for which the small-loss assumption (corrupted examples have larger loss) is expected to hold comfortably; instance-dependent or asymmetric noise is not studied, nor is real annotator noise.
- Oracle forget rate. The small-loss system is given the true noise rate. The comparison with CE, LS and mixup is therefore not a comparison of equally informed methods; the practical value of the rule depends on estimating r .
- Reimplementations, not reproductions. All four systems were reimplemented in one script. The selection rule is one-network and per-batch, not two-network Co-teaching; no tuning was done for any system in the main grid, and the sweeps show that the pre-set mixup α and the warm-up schedule were not optimal. A tuned comparison (equal tuning budget, validation set with noisy labels) was not run.
- No early stopping / no validation. We report the final epoch, which exaggerates memorisation relative to what a practitioner using early stopping on a clean validation set would see; the 5-epoch test-accuracy curves in the run logs show that the ranking at final epoch is specific to final-epoch evaluation; those checkpoints are selected on the test split and therefore optimistic, and their accuracies are not registry metrics, so no values are quoted.
- Test-split sweeps. The sensitivity sweeps are evaluated on the test split. They describe the response surface and should not be read as tuned results.
- Compute accounting. Runs shared a loaded machine; median wall time per run was about 23s but the two slowest runs took 334s and 367s because of contention, not because of the method.
- Identical behaviour at 0%. The small-loss system equals CE at 0% noise by construction, so no information about its cost on clean data (where an estimated, non-zero rate would be used) is obtained.

VIII. Conclusion

On 8×8 digits with a 41k-parameter CNN trained to the final epoch, plain CE memorised essentially all corrupted labels and fell from 0.991 to 0.709 test accuracy at 40% symmetric noise; label smoothing did not help at the final

epoch and was slightly worse at 40% (it exceeded CE at earlier logged epochs); mixup ($\alpha=1$) helped modestly while still memorising about 90% of corrupted labels; and small-loss selection with the true noise rate was clearly best (0.896 vs 0.709 for CE at 40%) and the only method to classify most corrupted training examples by their true class. The selection rule’s success depends on the assumed forget rate (under-estimation is costly, over-estimation to 0.6 was not), and larger mixup α gave progressively better accuracy, so the pre-set $\alpha=1$ understates mixup. These results are from one small dataset, five seeds and untuned baselines; a fuller study would use larger datasets, deeper networks, tuned and equally budgeted baselines with a noisy validation set, estimated noise rates, and non-symmetric noise.

References

- [1] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, “Understanding deep learning requires rethinking generalization,” arXiv preprint arXiv:1611.03530, 2016. [Online]. Available: <https://arxiv.org/abs/1611.03530>
- [2] D. Arpit, S. Jastrzebski, N. Ballas, D. Krueger, E. Bengio, M. S. Kanwal, T. Maharaj, A. Fischer, A. Courville, Y. Bengio, and S. Lacoste-Julien, “A Closer Look at Memorization in Deep Networks,” arXiv preprint arXiv:1706.05394, 2017. [Online]. Available: <https://arxiv.org/abs/1706.05394>
- [3] L. Jiang, Z. Zhou, T. Leung, L.-J. Li, and L. Fei-Fei, “MentorNet: Learning Data-Driven Curriculum for Very Deep Neural Networks on Corrupted Labels,” arXiv preprint arXiv:1712.05055, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05055>
- [4] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama, “Co-teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels,” arXiv preprint arXiv:1804.06872, 2018. [Online]. Available: <https://arxiv.org/abs/1804.06872>
- [5] J. Li, R. Socher, and S. C. H. Hoi, “DivideMix: Learning with Noisy Labels as Semi-supervised Learning,” arXiv preprint arXiv:2002.07394, 2020. [Online]. Available: <https://arxiv.org/abs/2002.07394>
- [6] S. Liu, J. Niles-Weed, N. Razavian, and C. Fernandez-Granda, “Early-Learning Regularization Prevents Memorization of Noisy Labels,” arXiv preprint arXiv:2007.00151, 2020. [Online]. Available: <https://arxiv.org/abs/2007.00151>
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” arXiv preprint arXiv:1710.09412, 2017. [Online]. Available: <https://arxiv.org/abs/1710.09412>
- [8] R. Müller, S. Kornblith, and G. Hinton, “When Does Label Smoothing Help?” arXiv preprint arXiv:1906.02629, 2019. [Online]. Available: <https://arxiv.org/abs/1906.02629>
- [9] D. Patel and P. S. Sastry, “Memorization in Deep Neural Networks: Does the Loss Function matter?” arXiv preprint arXiv:2107.09957, 2021. [Online]. Available: <https://arxiv.org/abs/2107.09957>
- [10] M. Lukasik, S. Bhojanapalli, A. K. Menon, and S. Kumar, “Does label smoothing mitigate label noise?” arXiv preprint arXiv:2003.02819, 2020. [Online]. Available: <https://arxiv.org/abs/2003.02819>
- [11] J. Wei, H. Liu, T. Liu, G. Niu, M. Sugiyama, and Y. Liu, “To Smooth or Not? When Label Smoothing Meets Noisy Labels,” arXiv preprint arXiv:2106.04149, 2021. [Online]. Available: <https://arxiv.org/abs/2106.04149>